Designing Resilient Data Pipelines

Modern analytics teams depend on pipelines that move data from dozens of sources into a single warehouse. When one of those pipelines fails quietly, every dashboard downstream shows numbers that look plausible but are wrong. This article describes the practices we adopted after a year of incidents, and the trade-offs behind each of them.

Why pipelines fail

Most failures we saw were not dramatic crashes. They were *silent* changes: a vendor renamed a column, a time zone shifted by one hour, or a partner started sending duplicate rows. The job still finished with a green status, so nobody looked at it until a finance analyst noticed that revenue had doubled overnight.

We grouped the root causes into four families:

- **Schema drift**, where the shape of incoming data changes without notice
- **Volume anomalies**, such as a feed that suddenly delivers zero rows or ten times the usual amount
- **Semantic errors**, where values are valid but mean something different
  - currency codes that switched from cents to dollars
  - status fields that gained a new value
- **Infrastructure faults**, including expired credentials and full disks

Principles we follow

Validate at the boundary

Every record is checked when it enters the system, not when it reaches a report. The checks are cheap and declarative, and a failed check stops the load before bad data spreads. We keep the rules next to the code that reads the source, so a reviewer sees both in the same change.

1. Reject rows whose required fields are empty.

2. Compare the row count with a rolling seven-day median.
3. Quarantine, rather than drop, any record that fails a rule.
4. Alert the owning team with a link to the quarantined batch.

Make every step idempotent

A step that can run twice without changing the result is a step you can retry safely. We write each output partition in full and swap it in atomically, instead of appending to a table that may already hold part of the run.

> *Idempotence turned our worst incidents into routine reruns. Before, a retry meant a manual cleanup; after, it meant pressing a button.*

Keep the configuration small

Configuration files tend to grow until nobody understands them. We limit each pipeline to a short file that names its source, its schedule and its owner, and we keep defaults in one shared module.

```
from pipeline import Source, schedule

orders = Source(
    name="orders",
    owner="commerce-data",
    schedule=schedule.daily(hour=2),
    checks=["not_null:order_id", "row_count:7d_median"],
)
```

Measuring the results

We tracked three numbers for six months before and after the change. The table below summarizes them.

Metric	Before	After	Change
Incidents per month	14	3	-79%
Mean time to detect	31 h	2 h	-94%

Metric	Before	After	Change
Mean time to recover	6.5 h	0.8 h	-88%
Manual reruns	22	4	-82%

The largest gain came from detection. Problems that once surfaced days later were now caught within the same hour, often before anyone downstream had opened a report.

Lessons for other teams

Start with the checks that would have caught your last three incidents, not with a framework. A handful of well-placed rules does more than a large library that nobody tunes. Name an owner for every source, and make the alert reach that person directly. Finally, treat the quarantine as a product: if it is hard to inspect, people will bypass it.

For further reading, see the data quality guide and the notes on `retry_policy` in the platform handbook.